

Ex.No:	
DATE:	

AIM:

ALGORITHM:

CODE:

seed = 3

a = 5

c = 2

m = 13

n = 5

```

sequence = [seed]

x = seed

print("Linear Congruential Generator")

print(" ")

for i in range(1, n + 1):

    x = (a * x + c) % m

    sequence.append(x)

    print(f"X{i} = {x}")

print("\nGenerated Sequence:")

print(sequence)

if sequence[-1] in sequence[:-1]:

    print("Sequence Repeats: Yes")

    period = len(sequence) - 1 - sequence.index(sequence[-1])

    print("Period =", period)

else:

    print("Sequence Repeats: No")

```

OUTPUT:

```

Linear Congruential Generator
-----
X1 = 4
X2 = 9
X3 = 8
X4 = 3
X5 = 4

Generated Sequence:
[3, 4, 9, 8, 3, 4]
Sequence Repeats: Yes
Period = 4

```

RESULT:

Ex.No:	
DATE:	

AIM:

ALGORITHM:

CODE:

```
import math  
p = 7  
q = 11  
seed = 16  
num_bits = 5  
n = p * q
```

```
if math.gcd(seed, n) != 1:
    print("Invalid Seed")
else:
    x = seed
    bits = []
    print("Blum Blum Shub Generator")
    print(" ")
    for i in range(num_bits):
        x = (x * x) % n
        bit = x % 2
        bits.append(bit)
        print("Random Number =", x, " Output Bit =", bit)
    print("\nBit Stream =", "".join(map(str, bits)))
```

OUTPUT:

```
Blum Blum Shub Generator
-----
Random Number = 25  Output Bit = 1
Random Number = 9   Output Bit = 1
Random Number = 4   Output Bit = 0
Random Number = 16  Output Bit = 0
Random Number = 25  Output Bit = 1

Bit Stream = 11001
```

RESULT:

Ex.No:	
DATE:	

AIM:

ALGORITHM:

CODE:

```
register = [1, 0, 1, 1]
n = 10
print("Linear Feedback Shift Register")
print(" ")
for i in range(n):
```

```
feedback = register[0] ^ register[3]
output = register[-1]
register = [feedback] + register[:-1]
print("Step", i + 1)
print("Feedback =", feedback)
print("Output =", output)
print("Register =", register)
print()
```

OUTPUT:

Linear Feedback Shift Register

Step 1
Feedback = 0
Output = 1
Register = [0, 1, 0, 1]

Step 2
Feedback = 1
Output = 1
Register = [1, 0, 1, 0]

Step 3
Feedback = 1
Output = 0
Register = [1, 1, 0, 1]

Step 4
Feedback = 0
Output = 1
Register = [0, 1, 1, 0]

Step 5
Feedback = 0
Output = 0
Register = [0, 0, 1, 1]

Step 6
Feedback = 1
Output = 1
Register = [1, 0, 0, 1]

Step 7
Feedback = 0
Output = 1
Register = [0, 1, 0, 0]

Step 8
Feedback = 0
Output = 0
Register = [0, 0, 1, 0]

Step 9
Feedback = 0
Output = 0
Register = [0, 0, 0, 1]

Step 10
Feedback = 1
Output = 1
Register = [1, 0, 0, 0]

RESULT: